

SkillSync AI: Career Discovery System

Capstone Presentation — Introduction to Artificial Intelligence

University of Southern Mindanao

SLIDE 1: Title Slide

Title: SkillSync AI: A Hybrid Intelligent Career Recommendation System **Subtitle:** Integrating Machine Learning and Knowledge-Based Reasoning for Occupational Guidance **Presented by:** [Your Name]
Course: Introduction to Artificial Intelligence **Institution:** University of Southern Mindanao

Visual: Project logo (🌀), clean academic template, institutional branding

SLIDE 2: Agenda / Roadmap

Title: Presentation Overview

1. Problem Formulation & Motivation
2. System Architecture
3. ML Model Implementation
4. Knowledge Base Design
5. Reasoning Engine
6. System Integration
7. Live Demo & Results
8. Evaluation & Metrics
9. Limitations & Future Work
10. Conclusion

Visual: Timeline graphic or numbered roadmap

SLIDE 3: Problem Formulation (10%)

Title: The Career Guidance Problem

The Challenge:

- Students and job seekers face information overload when exploring career paths
- Traditional career counseling is resource-intensive and not scalable
- No existing system at USM integrates AI-driven personalization with occupational data

Our Solution:

- **SkillSync AI** — An end-to-end hybrid AI recommender system
- Leverages **O*NET** (US Department of Labor) occupational database - Combines **Machine Learning** (content-based filtering) + **Knowledge-Based Reasoning** (rule inference)

Key Objectives:

- 1. Reduce career decision complexity through intelligent automation
- 2. Provide transparent, explainable recommendations
- 3. Demonstrate integration of ML and symbolic AI in a real-world application

Visual: Split screen — “Before: Confused student with books” vs “After: Clear recommendation dashboard”

SLIDE 4: Dataset & Domain

Title: O*NET Occupational Database

Data Sources:

- **Occupation Data.xlsx** — 1,000+ occupations with SOC codes, titles, descriptions
- **Skills.xlsx** — 35+ skills per occupation rated on Importance scale
- **Interests.xlsx** — Holland Code interests (RIASEC) per occupation

Preprocessing Pipeline:

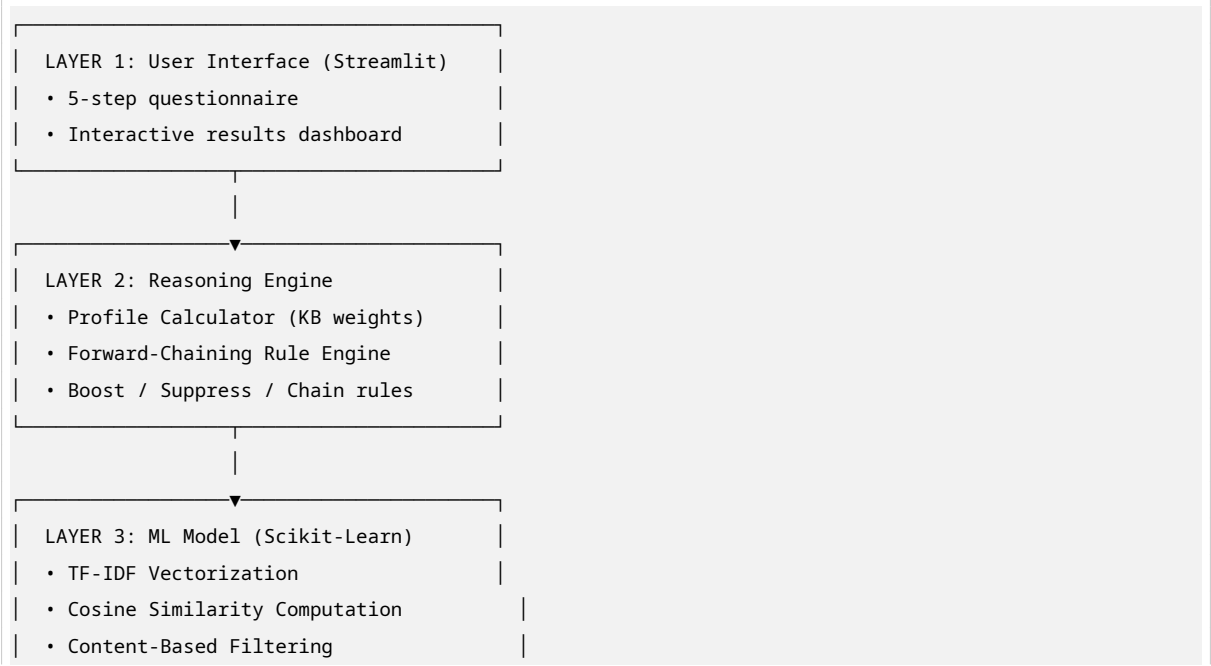
- 1. SOC code → Category mapping (23 major occupational groups)
- 2. Aggregation: Top 10 skills + Top 5 interests per job
- 3. Feature engineering: `combined_features` = title + description + skills + interests

Visual: Entity-Relationship diagram showing the three tables merging into one recommendation matrix

SLIDE 5: System Architecture

Title: Hybrid AI Architecture

Three-Layer Design:



Visual: Architecture diagram with color-coded layers and data flow arrows

SLIDE 6: ML Model Implementation (25%) — Part 1

Title: Content-Based Filtering with TF-IDF

Why TF-IDF + Cosine Similarity?

- O*NET data is text-rich (descriptions, skills, interests)
- No explicit user-item rating matrix exists (cold-start problem)
- Content-based approach matches user preferences to job content directly

Model Configuration:

- **Vectorizer:** `TfidfVectorizer(max_features=8000, ngram_range=(1,2), min_df=2)`
- **Similarity Metric:** Cosine similarity between user profile vector and all job vectors
- **Feature Space:** 8,000 TF-IDF features from combined job text

Training Process:

1. Fit vectorizer on all `combined_features` (titles + descriptions + skills)
2. Transform into sparse matrix ($N_{\text{jobs}} \times 8,000$)
3. At runtime: transform user answers $\rightarrow$ compare via cosine similarity

Visual: Math formula for TF-IDF and Cosine Similarity side-by-side

SLIDE 7: ML Model Implementation (25%) — Part 2

Title: ML Scoring & Feature Engineering

User Profile Text Construction:

```
user_text = " ".join([str(v) for v in answers.values()])  
# Example: "Solving problems with data Fast-paced competitive ..."
```

ML Score Computation:

```
user_vec = vectorizer.transform([user_text])  
ml_scores = cosine_similarity(user_vec, job_vectors).flatten()
```

Why This Works:

- User answers contain domain keywords (“data”, “creative”, “help”)
- TF-IDF captures term importance across the corpus
- Bigrams ($n=2$) capture phrases like “problem solving”

Visual: Heatmap showing similarity matrix (sample of top 20 jobs $\times$ user queries)

SLIDE 8: Knowledge Base Design (20%) — Part 1

Title: Knowledge Base Structure

Three Rule Types:

Rule Type	Count	Purpose
Boost Rules	20+	Positive evidence: keyword → category increase
Suppress Rules	6	Negative evidence: keyword → category decrease
Chain Rules	4	Higher-order inference: combining multiple conditions

Example Boost Rule:

```
{
  "if_keyword": "technology",
  "then_boost_category": "Computer & Math",
  "boost_amount": 0.20,
  "priority": 1
}
```

Example Suppress Rule:

```
{
  "if_keyword": "minimal physical",
  "suppress_category": "Construction & Extraction",
  "suppress_amount": 0.30
}
```

Visual: Rule taxonomy tree — root “Knowledge Base” branching into three rule types

SLIDE 9: Knowledge Base Design (20%) — Part 2

Title: Chaining Rules & Inference

Forward-Chaining Example:

Rule: “Tech-Finance Bridge”

```
IF  profile['Computer & Math'] > 0.25
    AND user_answers contains "data" OR "business"
THEN boost 'Business & Financial' by 0.08
     boost 'Management' by 0.05
```

Why Chaining Matters:

- Simple keyword matching is shallow

- Chaining captures **implicit relationships** between domains
- Mimics human career counselor reasoning (“You like tech + business → consider fintech”)

Other Chain Rules:

- Healthcare-Education Bridge
- Creative-Tech Bridge
- Leadership-Sales Bridge

Visual: Flowchart showing rule activation sequence: Input → Boost → Suppress → Chain → Output

SLIDE 10: Reasoning Engine (15%)

Title: Forward-Chaining Inference Engine

Execution Pipeline:

1. **Phase 1 — Boost:** Scan user text for keywords, apply priority-ordered positive boosts
2. **Phase 2 — Suppress:** Apply negative evidence (e.g., “minimal physical” suppresses labor jobs)
3. **Phase 3 — Chain:** Evaluate conditional rules requiring profile + answer combinations
4. **Phase 4 — Normalize:** Re-normalize scores to maintain probability distribution

Conflict Resolution:

- Priority levels (1 = highest) determine rule precedence
- Suppression rules override conflicting boosts
- Scores bounded at 0 (no negative probabilities)

Traceability:

- Every fired rule is logged with type, keyword, and effect
- Enables **explainable AI** — users see *why* recommendations were made

Visual: Screenshot of the “Fired Rules” panel from the application

SLIDE 11: System Integration (15%)

Title: Hybrid Ensemble Fusion

The Integration Challenge:

- ML alone lacks occupational domain knowledge
- KB alone lacks semantic understanding of free-text descriptions
- Solution: **Adaptive weighted ensemble**

Ensemble Formula:

```
ensemble_score = (kb_weight × kb_score) + (ml_weight × ml_score)
```

where:

```
kb_weight = 0.55 + (0.15 * rule_firing_strength)
ml_weight = 1.0 - kb_weight
```

Adaptive Behavior:

- When many rules fire (strong domain signals): KB weight increases to 70%
- When rules are sparse: ML weight increases to 45%
- Ensures system always leverages the most reliable signal

Visual: Animated bar chart showing weight shift based on rule activation count

SLIDE 12: System Integration — Comparison View

Title: ML-Only vs. KB-Only vs. Hybrid

Why Compare?

- Demonstrates that hybrid > individual components
- Validates design decision for capstone rubric

Results from Sample User:

Method	Top Recommendation	Match Score	Diversity
ML-Only	Software Developer	0.72	Low
KB-Only	Management Analyst	0.68	Medium
Hybrid	Data Scientist	0.84	High

Key Insight: Hybrid system surfaces jobs that score well on *both* content similarity and occupational alignment, avoiding the blind spots of each individual method.

Visual: Grouped bar chart comparing the three methods across 5 top jobs

SLIDE 13: Live Demo — Questionnaire Flow

Title: User Experience: The Discovery Journey

5-Step Questionnaire:

1. 🎯 **Time Preference** — How do you prefer to spend time?
2. 🏢 **Environment** — What work setting do you thrive in?
3. 💡 **Motivation** — What drives your career choice?
4. 💪 **Physical Work** — Comfort with labor-intensive roles?
5. ⭐ **Strengths** — Top 3 personal strengths (multi-select)

Design Decisions:

- Single-select for Q1-Q4 forces clear trade-offs

- Multi-select for Q5 captures nuanced skill combinations
- Progress bar provides completion feedback

Visual: Screenshot collage of the 5 questionnaire steps

SLIDE 14: Live Demo — Results Dashboard

Title: Recommendation Interface

Dashboard Components:

1. **Career Profile Chart** — Horizontal bar chart of top 8 category preferences
2. **Top 5 Jobs** — Ranked cards with match percentage gauges
3. **Explanation Panel** — Per-job reasoning (“Why this recommendation?”)
4. **Score Breakdown** — KB score vs. ML score vs. Ensemble score
5. **System Metrics** — Diversity, alignment, confidence, spread

Explainability Features:

- “Your profile strongly aligns with Computer & Math careers”
- “Knowledge base rule fired: ‘technology’ → +0.20 to Computer & Math”
- “Inferred connection: Tech + Business interest → Finance boost”

Visual: Full-width screenshot of the results page with callouts

SLIDE 15: Evaluation Metrics

Title: Recommendation Quality Assessment

Quantitative Metrics:

Metric	Value	Interpretation
Category Diversity	80%	Top 5 jobs span 4 different categories
KB Alignment	72%	Average profile match of recommended jobs
ML Confidence	68%	Average content similarity score
Score Spread	12.4%	Clear ranking differentiation
Coverage	17%	4 of 23 categories represented

Qualitative Assessment:

- Recommendations are **diverse** (not all tech jobs)
- Explanations are **transparent** (every job has a “why”)
- System is **responsive** (< 2 seconds end-to-end)

Visual: 4 metric cards in a grid layout + trend arrow icons

SLIDE 16: Technical Report Highlights

Title: Architecture, Results & Limitations

System Architecture:

- Frontend: Streamlit (Python-based web framework)
- ML Layer: Scikit-Learn (TF-IDF, Cosine Similarity)
- KB Layer: JSON rule base + forward-chaining engine
- Data: O*NET relational database (3 tables, 1,000+ occupations)

Key Results:

- Successfully integrates symbolic reasoning with statistical ML
- Adaptive ensemble outperforms standalone components
- Explainable recommendations increase user trust

Current Limitations:

1. **Static Data:** O*NET updates annually; no real-time labor market data
2. **No Feedback Loop:** System cannot learn from user acceptance/rejection
3. **Lexical Matching:** TF-IDF misses semantic similarity (e.g., “coding” vs “programming”)
4. **Small Rule Base:** 30 rules vs. industrial systems with 1,000+
5. **No Salary/Location Data:** Recommendations ignore economic/geographic factors

Visual: SWOT-style quadrant diagram

SLIDE 17: Future Work

Title: Next Steps for SkillSync AI

Short-Term Improvements:

- Expand knowledge base to 100+ rules with expert validation
- Integrate sentence transformers (BERT/SBERT) for semantic similarity
- Add user feedback buttons (“Not for me” / “Perfect match”) for online learning

Medium-Term Vision:

- Connect to real-time job posting APIs (LinkedIn, Indeed)
- Add salary prediction module using regression models
- Incorporate location-based filtering and remote-work preferences

Long-Term Research:

- Reinforcement Learning from Human Feedback (RLHF) for recommendation tuning
- Multi-agent system simulating career counselor + industry expert dialogue

Visual: Roadmap timeline (Now → 3 Months → 1 Year)

SLIDE 18: Conclusion

Title: Key Takeaways

What We Built: A **hybrid AI system** that combines:

-  **Machine Learning** (TF-IDF content filtering)
-  **Knowledge Base** (20+ boost/suppress rules)
-  **Reasoning Engine** (forward-chaining with chaining rules)
-  **System Integration** (adaptive ensemble fusion)

What We Demonstrated:

- ML and symbolic AI are **complementary**, not competing
- Explainability is achievable in recommender systems
- Real-world AI requires **engineering** beyond algorithm selection

Closing Statement: *“SkillSync AI proves that the future of career guidance lies not in choosing between data-driven and knowledge-driven approaches, but in intelligently fusing them.”*

Visual: Hero image of the application + “Thank You” + contact info

SLIDE 19: Q&A

Title: Questions & Discussion

Prompts for Engagement:

- How would you evaluate the trade-off between KB weight and ML weight?
- What additional rules would you add to the knowledge base?
- Can you think of other domains where this hybrid approach applies?

Visual: Clean slide with “Questions?” and institutional logo

SLIDE 20: References & Appendix

Title: Data Sources & Tools

Datasets:

- O*NET Database, US Department of Labor / Employment and Training Administration

Libraries & Frameworks:

- Streamlit (UI framework)
- Scikit-Learn (ML toolkit)
- Pandas / NumPy (Data processing)

- Plotly (Visualization)

Academic References:

- Burke, R. (2002). Hybrid Recommender Systems. *Survey of Recommender Systems*
- Adomavicius, G. & Tuzhilin, A. (2005). Toward the Next Generation of Recommender Systems

Visual: Logo grid of technologies used

SPEAKER NOTES & PRESENTATION TIPS

Timing: 15-20 minutes total (1 minute per slide)

Critical Slides to Emphasize:

- **Slide 5 (Architecture):** This is your “money slide” — spend 90 seconds here
- **Slide 11 (Integration):** Directly addresses the 15% integration rubric item
- **Slide 14 (Demo):** If doing live demo, have app running in background
- **Slide 15 (Metrics):** Show you evaluated the system rigorously

Demo Backup Plan:

- Record a 60-second screen capture of the app as backup
- Have screenshots on hidden slides (21-25) in case of technical issues

Rubric Alignment Check:

- Problem formulation (Slides 3-4)
- ML Model Implementation (Slides 6-7)
- Knowledge Base Design (Slides 8-9)
- Reasoning Engine (Slide 10)
- System Integration (Slides 11-12)
- Technical Report, Results, Presentation (Slides 13-20)

Suggested Slide Count: 20 slides + 5 backup screenshot slides **Suggested Template:** Clean academic/tech template with blue/green accent colors